Technical University of Crete
School of Electrical and Computer Engineering
Course: Randomized Algorithms
Exercise 3 (100/1000)
Report Delivery Date: 9/12/2024
Instructor: Athanasios P. Liavas

Paraskevopoulou Despoina
2021030135

Max Cut Problem

In the first part of this exercise we studied the Large-Cut algorithm in random graphs. We simulated the problem in two phases, first implementing the randomized Large Cut algorithm and second the deterministic algorithm.

1 . First, we implemented the randomized Large-Cut algorithm for random graph $G_{n,p}$ where n is the number of nodes and p is the probability that any pair of nodes is connected. Below, is presented the algorithm we used:

Algorithm 1 Randomized Large-Cut

```
while iter < m/2 + 1 do
    for every node v do
        insert v in A with probability p
        or
        insert v in B with probability 1-p
    end for
    for every element in A do
        for every element in B do
            compute large-cut
        end for
    end for
    if large-cut > m/2 then
        Large-Cut found
    end if
end while
```

2. Through the process of de-randomization arises the deterministic version of the above algorithm which we present below:

Algorithm 2 Deterministic Large-Cut

```
v_1 ← A(1)
for every node [v_2...v_n] do
    count the neighbors of A and B
    place v_i on the side with fewer neighbors
end for
for every element in A do
    for every element in B do
        compute large-cut
    end for
end for
```

We implemented the two algorithms in the same graph. What we observed is that the large-cut derived from the deterministic algorithm is larger, which was an expected result. The result from the randomized algorithm is usually close to the number $\frac{\#\text{edges}}{2}$, which was also expected, having studied the theory. The deterministic algorithm gives a better result because it does a greedy approach, so we can expect it to give a larger cut. An example of the difference between the results of those two algorithms is given below, for $n = 1000$ and $p = 0.01$:

```
Large Cut should be at least: 2494
Randomized large cut is : 2517, found in 2 iterations
Deterministic large cut is : 3335
```

Figure 1: Randomized and Deterministic Algorithm Results for Large-Cut

Balls and Bins

In the second part of this exercise, we proceeded to study the concentration of the number of empty bins around its mean, in the m-Balls and n-Bins experiment. We begin by throwing m-balls into n-bins, uniformly at random, and we counted the empty bins. We repeated this experiment for K times. From the theory we know that the statistical mean of the empty bins is given as:

$$E[F] = n(1 - \frac{1}{n})^m$$

where F represents e the number of empty bins after m balls are thrown to n bins.
Then, we moved on to plot the histogram of the number of empty bins in the interval $[n_{min} : 1 : n_{max}]$, for example from 0 to 499, because we can have the extreme cases of no empty bin, or one empty bin. We observe that the concentration around the mean is really dense. If we zoom in the plot we see a resemblance to the Gaussian distribution. All the values of the means over K experiments are distributed around the arithmetic mean, which is pretty close to the statistical mean.
Below, we present some results:

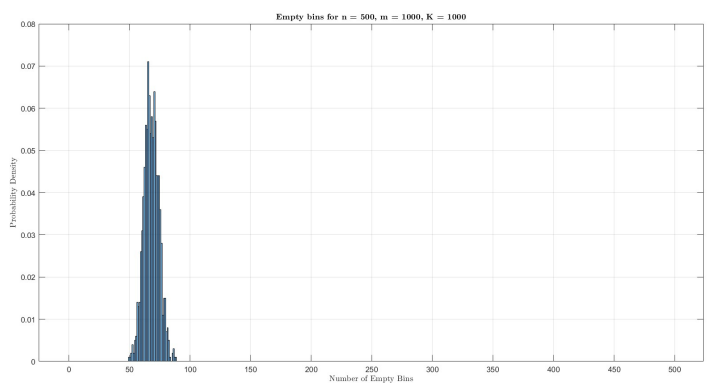

Figure 2: 1000 balls thrown in 500 bins over 1000 experiments

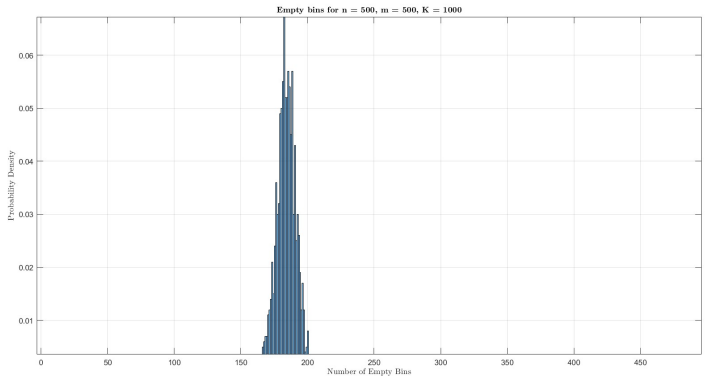

Figure 3: 500 balls thrown in 500 bins over 1000 experiments

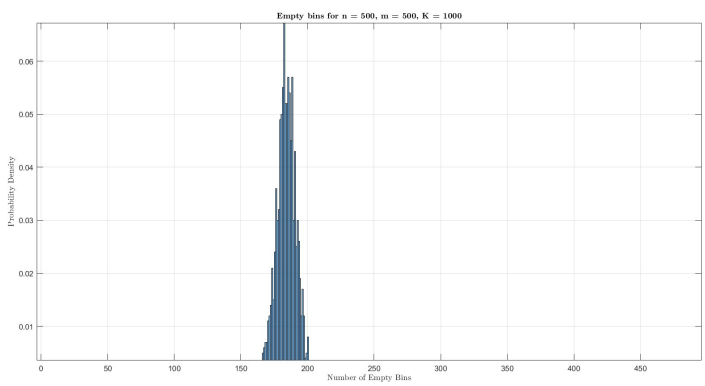

Figure 4: 1000 balls thrown in 1000 bins over 1000 experiments

We observe that indeed the plots are a resemblance to the Gaussian distribution and all the arithmetic mean values of the K experiments are concentrated around a mean value which is pretty close to the statistical, almost the same.

Appendix

Below we represent the code that was used for this exercise.

```
function [max_cut,iter] = max_cut_Random(n,G,m)
    iter = 0; cut_found = false;

    while(iter < m/2 +1)
        A = []; B = [];
        max_cut=0;
        iter = iter+1;

        for v=1:n
            if rand() < 0.5
                A = [A,v];
            else
                B = [B,v];
            end
        end

        for i=A
            for j=B
                max_cut = max_cut + G(i,j);
            end
        end

        if (max_cut>=m/2)
            cut_found = true;
            break;
        end
    end
end
```

```
function max_cut_det = max_cut_deter(n,G,A,B,m,order)
    cutD_found = false;
    max_cut_det = 0;
    A = [A,order(1)];

    for v=2:n
        n_in_A = sum(G(v,A)); % neighbors in A
        n_in_B = sum(G(v,B));

        if n_in_A <= n_in_B
            A = [A,v];
        else
            B = [B,v];
        end
    end

    for i=A
        for j=B
            max_cut_det = max_cut_det + G(i,j);
        end
    end

    if (max_cut_det>=m/2)
        cutD_found = true;
    end

end
```

```
function empty_bins = balls_bins_empty(n,m,K)
    B = zeros(1,K);
    empty_bins = zeros(1,K);

    for i = 1:K
        bins = zeros(1,n);
        ball_i = 0;

        for j=1:m
            b=randi(n);
            bins(b) = bins(b)+1;
            ball_i = ball_i + 1;
        end

        for bin=1:n
            if bins(bin) == 0
                empty_bins(i) = empty_bins(i) + 1;
            end
        end
    end
end
```